Cumulative Review

Type Annotation & Type Inference

What are type annotation & type inference?

Type annotation allows us to explicitly state the type of a variable or constant. Type inference is used when we do not specify a type, this allows the compiler to do the work for us.

Typically you should prefer using type inference, less code is better code. There are instances where you need to use type annotation. Regardless of if you are annotating or inferring a variable or constants type, it is important to know the existing types, and how to use them.

Type examples

So here are some examples of each of the types we have covered with type annotation. You have already seen functions, and those are the first time where you cannot get away without knowing how to write these types.

```
let stringTypeExample: String = ""
let intTypeExample: Int = 0
let doubleTypeExample: Double = 0.0
let boolTypeExample: Bool = true
```

We also have our collection types and tuples. These are a little harder to understand, and get used to as there are multiple pieces. Our collections are all compounded types, meaning we specify the type of collection we are using, as well as the type of information they store.

```
// Arrays
let arrayOfStringTypeExample: [String] = []
let arrayOfIntTypeExample: [Int] = []
let arrayOfBoolTypeExample: [Bool] = []

// Sets
let setOfIntTypeExample: Set<Int> = []
let setOfStringTypeExample: Set<String> = []
```

We also have Dictionaries and Tuples which have multiple types that we have to specify. Dictionaries take two types, one for the key and one for the value. Tuples take as many

types as we want, where we have one type per value included in the tuple.

```
// Dictionaries
let keyStringValueIntDictionaryTypeExample: [String: Int] = [:]
let keyDoubleValueBoolDictionaryTypeExample: [Double: Bool] = [:]

// Tuples
let tupleWithIntAndDoubleExample: (Int, Double) = (1, 2.5)
let tupleWithStringAndBoolTypeExample: (String, Bool) = ("Hello",
false)

// Tuple with Labels
let rgbTupleTypeExample: (red: Int, green: Int, blue: Int) = (128, 0,
128)
```

It is important to remember that though our collections and tuples are compounded, they are still Types, meaning we can treat them the same as our basic types. For example, we can have an Array of Arrays, or an Array of Dictionaries. We could even have a Dictionary with a value that is an Array of Tuples. They are all just types at the end of the day!

```
let keyDoubleSetDictionaryTypeExample: [Double: Set<Bool>] = [:]
let arrayOfArrayOfIntegers: [[Int]] = []
let arrayOfKeyStringValueDoubleDictionary: [[String: Double]] = []
let keyIntValueDictionaryWhereKeyStringValueDouble: [Int: [String:
Double]] = [:]
```

Collections

So we have gone back over Type annotation, and Type inference, and how it applies to our collections and tuples, so now we are going to cover the semantics of them, rather than the Syntax. So in other words, why, not how. We will still do some examples, however. As a reminder there are 4 types of collections in Swift by default.

1. Arrays: A list of items that preserve order and can have duplicates.
2. Sets: An unordered list of unique items.
3. Dictionaries: A collection of key, value pairs.
4. Tuples: A fixed size collection of optionally named values, where each value can have a different type.

So now lets talk about when we might use each of these. Starting with the crowd favorite: Arrays.

Arrays

Arrays are good at storing a group of things, like a grocery list. They are used when you need to be able to have multiple items that are the same, and need easy access to the items. Arrays are also typically the go to collection if you don't need the features of one of the others.

Let's do a handful of examples of how an Array may be used.

```
// Shopping List
let myShoppingList = ["Apple", "TV Dinner", "Apple", "Cereal", "Water"]

// Tracking articles read by a specific user, where order matters.
let articleIDsReadByUser = [24573, 42890, 1256, 90083]
```

Sets

Sets are also great at storing groups of information, but only if you do not care about the order, and only want unique items. Let's say you are storing all the different words that you know. You wouldn't want to put a word in there twice by accident right?

```
let allTheWordsIKnow = Set([
    "I", "like", "to", "program", "I", "think",
    "it", "is", "fun", "to", "program"
])
```

Even though in the above example I put some words multiple times, since it is a Set the duplicates will be removed automatically.

Another example may be for keeping track of attendance using Student IDs. I may want to batch up the clock in information before sending it to a server, if a user scans multiple times, I only want to include one scan.

```
let attendanceIDs = Set([123, 234, 345, 126, 123])
```

Dictionaries

Dictionaries are great for dealing with information that needs to be easily and quickly identified. By using key, value pairs we can identify large amounts of information with easy identifiers. This avoids index ambiguity where we don't know exactly where an item in an Array is. A dictionary always has 2 types, the key type and the value type. In a dictionary the keys are always unique, however, the values do not have to be.

```
// Dictionary as a Dictionary
let myDictionaryOfTheWordsIKnowAndTheirDefinitions = [
```

```
    "programming": "The writing of a computer program.",
    "fun": "Enjoyment or amusement",
    "cool": "Lukas Simonson"
]

// Numbers and their Respective names.
let numbersAndTheirNames = [
    1: "One",
    2: "Two",
    3: "Three"
]

// States and their Populations
let statesAndTheirPopulations = [
    "Utah": 3_504_000,
    "Nevada": 3_267_000,
    "Texas": 31_290_000
]
```

Tuples

Tuples are used when you need to store a set number of related values. Those values can be different types. With tuples you cannot change the number of items stored in them. Tuples also allow us to provide names for the values we are storing. The main benefit of tuples is the ability to mix types in a single collection. Typically you should only have 2 to 3 values in a tuple, though there isn't a reasonable limit.

```
// One of the best and most common use for a Tuple
// is storing coordinates.
let myGridCoordinates = (x: 1, y: 2)

// Representing a Square with its name, position, and size.
let mySquareInfo = (name: "Cubert", x: 1, y: 10, size: 12.5, filled:
false)
```

Comparison Operators & Conditions

Comparison Operators / Expressions

Our comparison operators allow us to compare two values of the same type, and receive a `Boolean` value as a result of the comparison. There are 6 different comparison operators:

1. `>` : Greater than

2. `<` : Less than
3. `==` : Equal to, not to be confused with the `=` assignment operator.
4. `≥` : Greater than or equal to
5. `≤` : Less than or equal to
6. `≠` : Not equal to

When we use these operators we put them between two values, then that expression is evaluated, if the expression is true then the result is `true` else the result is `false`.

For example, if we created a constant and set it equal to one of these expressions, the type of the constant is a `Bool`.

```
let isOneLessThanTwo = 1 < 2 // true
```

If / else if / else

Booleans are very heavily used alongside various types of conditional statements / expressions in Swift. The most heavily used one is called an `if` statement. When we write `if` statements, we provide a boolean value / expression that dictates whether or not a given block of code is run. If the value is `true` the block of code runs, if the value is `false` it does not.

```
if true {
    print("This will always run")
}

if false {
    print("This will never run")
}
```

You will realistically never directly write `if true` or `if false`, but rather provide an existing boolean value, or more commonly, put a Boolean expression like the ones we just covered.

```
if 1 < 2 {
    print("One is less than two")
}

if 1 > 2 {
    print("One is greater than two")
}
```

Often times we want to also have code that runs when the value is true and when the value is false. We can add an `else` after the end of an `if` to run code only if the previous if's block did not run.

```
let isDarkMode = true

if isDarkMode {
    print("Show darkmode UI")
} else {
    print("Show lightmode UI")
}

let wordToCompareOne = "hi"
let wordToCompareTwo = "bye"

if wordToCompareOne == wordToCompareTwo {
    print("The words are the same")
} else {
    print("The words are different")
}
```

We can also include `else ifs` which have a condition to check, and only are checked if the previous conditions were all false. A statement / expression that uses these will always start with an `if` , and end with an `else` (if the else is present), `else if's` go between these two.

```
let firstNumberToCompare = 2
let secondNumberToCompare = 4

if firstNumberToCompare > secondNumberToCompare {
    print("First number is greater than second number")
} else if firstNumberToCompare < secondNumberToCompare {
    print("First number is less than second number")
} else {
    // We don't need an else if because if the numbers
    // aren't less than or greater than eachother they must be equal.
    print("First and second number are equal")
}
```

Logical Operators

Swift also has logical operators that allow us to combine or modify boolean values. These are often used as part of conditional statements to allow us further control over what

happens. For example, I may want to run the same block of code if one of two boolean values is true, or make sure that several boolean expressions are all true in order to do anything. There are 3 of these logical operators in Swift.

1. `||` : The double bar is the `OR` operator. If either side is true, the expression as a whole is true.
2. `&&` : The double ampersand is the `AND` operator. Both sides have to be true in order for this expression to be true.
3. `!` : The bang is the `NOT` operator. This operator provides the opposite of whatever expression it is attached to. A `true` expression with the `NOT` operator will equate to false.

Let's do an example where I have three different Integer constants. If at least two of the values are the same, I want to print `"At least two values are the same"`. If none of them are the same, then I want to print `"None of the values are the same"`.

```
let firstNumberToCompare = 5
let secondNumberToCompare = 2
let thirdNumberToCompare = 5

// If first number equals second number
// OR
// if first number equals third number
// OR
// if second number equals third number

if firstNumberToCompare == secondNumberToCompare ||
    firstNumberToCompare == thirdNumberToCompare ||
    secondNumberToCompare == thirdNumberToCompare
{
    print("At least two values are the same")
} else {
    print("None of the values are the same")
}
```

We could also flip this example using `≠` and the `AND` operator to get the same result.

```


// If first number doesn't equal second nubmer
// AND
// if first number doesn't equal third number
// AND
// if second number doesn't equal third number
```

```
if firstNumberToCompare ≠ secondNumberToCompare &&
    firstNumberToCompare ≠ thirdNumberToCompare &&
    secondNumberToCompare ≠ thirdNumberToCompare
{
    print("None of the values are the same")
} else {
    print("At least two values are the same")
}
```

The `NOT` operator or the bang, is used when we want the opposite of a boolean value. So it converts a `true` into a `false` and a `false` into a `true`. We use it by prefacing the value we want to reverse.

```
if !(1 > 2) {
    print("One is NOT greater than two")
}
```

Switch

Swift has another type of conditional operation we can utilize, this one instead of relying on Boolean values / expressions, it relies on equality checks. Meaning it is great for comparing one value to many different values. Typically if you are comparing a value against other values this is the way to go.

A switch has an extra requirement over an if, that being it must be exhaustive. We can rely on the `default` keyword to handle all non-covered cases, making this rather trivial.

Let's say for example, I have a dice roll, and based on that value I want to print a certain statement. We are comparing the result of the roll, to the numbers 1 through six, so it works great for a switch statement.

```
let diceRange = 1...6

switch Int.random(in: diceRange) {
    case 1: print("You rolled a one")
    case 2: print("You rolled a two")
    case 3: print("You rolled a three")
    case 4: print("You rolled a four")
    case 5: print("You rolled a five")
    case 6: print("You rolled a six")
    default: break
}
```

In the above example, we actually are doing something that we didn't cover just yet. When working with a switch we have to cover all possible cases; however, sometimes we don't want to do anything if there is another possible case. If we have that situation we can use the `break` keyword as part of our `default` to do nothing!

> **Note**
>
> In the above example, we know that the number will always be 1 through six; however, there is no way for the compiler to guarantee that, so we still need the `default`

Enumerations

Enumerations provide a way to represent a limited group of related values. They allow us an easy and safe way to represent this kind of information.

For example, you are building a game, and you need to define the different states that an enemy can be in. The enemy may be wandering, attacking, sitting still, or sleeping. Where there are a set number of states it can be in, an enumeration is a great way to represent this information.

```
enum EnemyState {
    case wander
    case attack
    case wait
    case sleep
}
```

When we declare an enum we are creating a new type, so the name is in upper camel case, and should also always be singular. It is singular because when we have a value of that type, it is only one of them. We can refer to the values created in the enum by using the name of the enum followed by a `.` and the case name.

```
let enemyCurrentState = EnemyState.wander
```

When working with enumerations, it is very common to use them alongside a switch. This is because where enumerations define a set number of options, switches are used to compare a set of values. This allows a really clean way to handle all the cases of an enumeration.

```
switch enemyCurrentState {
    case .wander:
        print("Just wandering around")
```

```
    case .attack:
        print("Attacking the player")
    case .wait:
        print("Waiting for something interesting to happen")
    case .sleep:
        print("Unconscious, perfect time to attack")

    // No default as all possible cases have been handled.
}
```

You can also check multiple cases with a switch statement by comma separating them. This allows you to run the same block of code for multiple cases. This works both with and without enumerations.

```
switch enemyCurrentState {
    case .wander, .attack:
        print("Doing something")
    default:
        print("Doing nothing")
}
```

Looping

Loops are a fundamental component in every programming language. They allow us to do repetitive tasks very easily. Whether we are wanting to do something with each item in a collection, do something a set number of times, or until a condition is met. Loops allow us to do it. Swift has a few types of loops, but the underlying mechanism behind them are the same. Run some code repeatedly.

For Loops

The most commonly used type of loop for our usage are for loops. These loop over Arrays and Ranges, each time the loop iterates it will pull out an item. This item may be an item in a collection, or a number in a Range. It will put this item into a constant that exists for the lifetime of that iteration.

```
// Using each number in a range.
for number in 0...10 {
    print(number)
}

// Using items in an Array.
let itemsToIterate = ["Item 0", "Item 1", "Item 2"]
```

```
for item in itemsToIterate {
    print(item)
}

// Performing a task, ignoring the provided value.
for _ in 0...10 {
    print("Ignored Value")
}
```

Let's do some examples from the looping assignment, or at least similar to it.

I need to print out hello world a number of times equal to a provided constant. Since we are doing something a set number of times, a for loop makes perfect sense.

```
let numberOfTimesToPrintHelloWorld = 10

for _ in 0..<numberOfTimesToPrintHelloWorld {
    print("Hello, World!")
}
```

There are a couple of things to point out here. When doing something `count` times, we need to still use a range. We cannot simply say `for _ in count`. We also used an `_` to ignore the provided constant. If you are not using that value you shouldn't provide a name for it.

Another more complicated question from the assignment would be as follows:

Print the first `numOfSquaresToPrint` square numbers. This is a more complex problem, so let's break it down into some comments, then solve the simpler problems.

```
let numOfSquaresToPrint = 6

// I need to get the first `numOfSquaresToPrint` square numbers.
// This means I need to do something that many times.
// Since we are doing something a set number of times, a for loop makes
sense.

// I need to know which number to square
// since we are counting up, we can use the constant provided by our for
loop
// I need to print something, but it doesn't specify it needs to be on a
single
// line so we can include the print directly in our for loop.
```

```
for squareMe in 1...numOfSquaresToPrint {
    print(squareMe * squareMe)
}
```

It is important to know how to both read and write code. It can be helpful to read code that you wrote pretending you didn't write it. This allows you to critique it, and walk through the code in a different light.

Let's try breaking down this syntax for the solution to the question. Most code that is written will start with a keyword of some kind. In this example our keyword is `for` so we know we are using a for loop to do something.

The next component is the name of the constant `squareMe`, since we didn't use an `_` we know that this constant is going to come into play in the for loop.

Next up is the `in` keyword, this is a standard part of any for loop, it really just pairs with `for`.

Now we have a range of numbers, `1...numOfSquaresToPrint`. This is an inclusive range, meaning it includes the upper bound number. We can tell it is inclusive because it uses `...` instead of `..<`.

Now inside the block of code we can see there is only one line: `print(squareMe * squareMe)`. We can see that we are multiplying the provided constant by itself, creating a square number, then printing the result.

While & Repeat Loops

A `for` loop is for when you know how many times you are looping. A `while` loop is for when you don't, instead you loop until a condition is false.

Similar to how an `if` statement contains a condition and then code that runs if that condition is true, a `while` loop just repeats that code until its condition is no longer true.

```
var number = 10

while number > 0 {
    print("The number is \(number)")
    number -= 1
}
```

⚠ Infinite Loops

> Unlike `for` loops, `while` loops can potentially run forever, crashing your code. It is your responsibility to make sure that the body of the `while` loop will eventually cause the condition to be false.

Lets look at a simple while loop, and then, without running it, figure out why it would break our code.

```
var otherNumber = 0
while otherNumber ≤ 10 {
    print(otherNumber)
    otherNumber -= 1 // Single character mistake causes a big issue.
}
```

In that example by simply using a `-` instead of a `+` our loop will continue infinitely.

Repeat loops are the same as a while loop with only 1 difference, the condition is checked AFTER the code has run. The only change this really makes, beyond syntax, is that a repeat loop always runs at least once, otherwise it is the same.

```
repeat {
    print("This statement is false")
} while false
```

Break & Continue

When we are in a loop we have two special keywords that we can use to escape the loop in some fashion. `break` & `continue`.

- `break`: Exits the current loop entirely.
- `continue`: Exits the current iteration, continuing the loop afterwords.

```
var characters = ["Finn", "Jake", "The Lich", "BMO"]

for character in characters {
    if character == "The Lich" {
        break // No longer loops.
    }

    print("\(character) is a Good Guy!")
}

// results:
```

```
// Finn is a Good Guy!
// Jake is a Good Guy!

for character in characters {
    if character == "The Lich" {
        continue // skips the rest of the current iteration.
    }

    print("\(character) is a Good Guy!")
}

// results:
// Finn is a Good Guy!
// Jake is a Good Guy!
// BMO is a Good Guy!
```

Functions

Functions are named self-contained chunks of code that do a single task. They can take parameters, which are named and typed inputs, and they can return a typed value output. `print` is an example of a function that we have used plenty of times. We call these functions by
invoking their name, followed by parentheses where we put the arguments, if there are any.

Declaring Functions

To declare a function we use the `func` keyword, followed by a name in lowerCamelCase, then the parentheses and curly braces for the body.

```
func helloWorld() {
    print("Hello, World!")
}

helloWorld()
```

Parameters

We can include named and typed parameters, which allow our functions to take in values. We do that by including, in the parentheses, a comma separated list of names and types, formated like `name: String`.

```
func greet(name: String) {
    print("Hello, \(name)")
```

```
}

greet(name: "Finn")
```

Swift also has a great feature called parameter labels, these change the name that is used when the function is called. Inside the function we still use the name, when it is called the label is used. If we use the special `_` label, that will make it so that no label is needed at all when passing our arguments.

```
func greet(_ name: String, with greeting: String = "Hello") {
    print("\(greeting), \(name)!")
}

greet("Jake", with: "Hi")
```

> **Arguments vs Parameters**
>
> Hopefully you are familiar with these terms already, but as a reminder there is a difference between arguments and parameters, even though we use them in very similar contexts. An argument is what gets passed to a function when we call it, so the actual values themselves. A parameter is what is defined by the function when it needs information.

We can also have default values for our parameters, this allows callers the ability to omit that specific argument, in that situation the default will be provided instead. We do that by adding the `=` assignment operator then the default value to provide.

Returning Values

A lot of times we want our functions to return values so that we can continue working with the results of the functions. To do this we need to specify that our function returns a value, we can do so by adding the `→ <Type>` syntax after our parameter list.

```
func greeting(for person: String) → String {
    return "Hello, \(person)!"
}

let result = greeting(for: "Simon") // Whats the type? → String
print(result)
```

Function Signatures

You may have noticed that we have multiple functions with the same name in these examples, but the compiler is not getting mad. This is because they each have their own function signatures. The function signature is what identifies each function in Swift. This signature is made up of the name, the parameter labels or names and their types, and the return type. As long as there is a difference in at least one of those then Swift can identify it as a unique function.

The trickiest part of this is remembering how the parameter labels and names are used. If there is a parameter label, it is used for the function signature, if there isn't one then the name is used.

```
func example() {}

// Can't exist, a function with the signature example() already exists.
// NOT ALLOWED
func example() {}

// Different parameters, allowed.
func example(with: String) {}

// Different parameter label, allowed.
func example(name with: String) {}

// Different name, but same parameter label, NOT ALLOWED.
func example(with name: String) {}

// Different parameter type, allowed.
func example(with: Int) {}

// Different return type, allowed.
func example() -> Int {
    return 0
}

// Different return type, allowed.
func example(with: String) -> Double {
    return 0
}
```

A common way to look at this is with a special write up of the function signature. This is not syntax, or even convention, it is just a common practice for documentation purposes.

```swift
// example()
func example() {}

// example(with:String)
func example(with: String) {}

// example(_:Int,_:Int)->Int
func example(_ first: Int, _ second: Int) -> Int {
    return 0
}

// example(x:Int,y:Int)->Int
func example(x: Int, y: Int) -> Int {
    return 0
}
```